CS683: Computer Architecture

Programming Assignment 2

Submitted by

Prathamesh Rajesh Thakur
26M0833

Jay Gupta
26M2101

Mohammed Nazal
26M0752

Rameez Wahid
26M0846

October 15, 2026

Task 1: Stride Pattern Prefetching (SPP)

October 14, 2026

Experimental Setup

All simulations use ChampSim (single core, hashed-perceptron branch predictor, LRU replacement at every level) with **25M warmup** and **25M simulation** instructions on the three provided traces. Only the L1D prefetcher is changed between configurations; every other cache level has no prefetcher.

- **Baseline**: no prefetcher anywhere.
- **IP-stride**: the provided `prefetcher/ip_stride.l1d_pref` (degree 3).
- **SPP**: our `prefetcher/spp.l1d_pref`.

Metrics are defined as in the assignment:

$$\text{Speedup} = \frac{\text{IPC}_{\text{prefetcher}}}{\text{IPC}_{\text{baseline}}}, \qquad \text{Coverage} = \frac{\text{Misses}_{\text{baseline}} - \text{Misses}_{\text{prefetcher}}}{\text{Misses}_{\text{baseline}}},$$

where *Misses* are L1D load misses. L1D load MPKI and prefetch accuracy are taken from ChampSim's statistics (accuracy = useful load prefetches / prefetches issued to the lower level). Raw outputs are in `results/`; tables and plots are produced by `report/task1/parse_task1.py`.

1 Task 1A: IP-Stride Prefetcher

The provided IP-stride prefetcher keeps a 24-entry fully associative tracker table with LRU replacement, indexed by the low 8 bits of the load IP. Each entry stores the last cache-line address accessed by that IP and the last observed stride. On every L1D access:

1. The IP is looked up. On a miss, the LRU entry is replaced and nothing is prefetched.
2. The stride (current line − last line) is computed. A stride of 0 is ignored.
3. If the new stride equals the stored stride (the same stride seen twice in a row), it prefetches lines $A + s, A + 2s, A + 3s$ (degree 3), stopping at the 4 KB page boundary. Prefetches fill into L1 when the MSHR is less than half full, otherwise into L2.
4. The entry's last address and stride are updated.

It only captures constant strides: any irregular but repeating sequence such as $(+1, +2, +2, +2, +3)^n$ never produces two equal consecutive strides often enough to trigger, which motivates SPP. Its results are included in the Task 1B tables as a reference point.

2 Task 1B: Stride Pattern Prefetcher (SPP)

2.1 Design

The design has two tables and a throttle:

IP table (64 entries, direct mapped). Indexed by `ip % 64` with an 8-bit partial tag. Each entry tracks the last line accessed by the IP, a *uniform* stride with a 2-bit confidence counter, and a 9-bit *signature* summarising the recent stride history.

Pattern table (512 entries, indexed by signature). Each entry maps a signature to the *next expected stride* with a 2-bit confidence counter. The signature is updated as

$$\text{sig}_{\text{new}} = \big((\text{sig} \ll 3) \oplus (\text{stride} \,\&\, \texttt{0x3F})\big) \,\&\, \texttt{0x1FF},$$

so it encodes roughly the last three strides. A repeating sequence such as $(+1, +2, +2, +2, +3)^n$ produces a repeating series of signatures, and each one learns which stride follows it.

Per-access operation.

1. **Lookup / allocate.** On a tag mismatch the entry is reset with the current line and no prefetch is issued.
2. **Page change.** If the access is in a different 4 KB page than the IP's last access, the history (stride, confidence, signature) is reset, because strides across pages are not meaningful in physical address space.
3. **Uniform training.** If the stride matches the stored stride, confidence is incremented (saturating at 3), otherwise decremented. The stored stride is replaced only once confidence reaches 0 (hysteresis).
4. **Non-uniform training.** The pattern-table entry for the current signature is trained with the observed stride in the same way, then the signature is updated.
5. **Dual pattern classification and prefetch.**
   - If uniform confidence ≥ 2, the IP is classified as **uniform** and we prefetch $A + s, A + 2s, \ldots, A + \text{degree} \cdot s$.
   - Otherwise it **falls back to non-uniform**: starting from the current signature, we walk the pattern table (lookahead), adding each predicted stride and updating a speculative signature, for up to *degree* steps. The walk stops at any entry with confidence < 1.

   Uniform prediction therefore has priority. When a constant stride breaks, its confidence counter decays to below 2 within a couple of accesses and the IP automatically switches to signature-based prediction. The switch back happens when the constant stride is seen again.
6. **Page boundary.** Every candidate prefetch is checked to be in the same page as the demand access, and the walk stops at the first one that would cross it. As in IP-stride, prefetches fill into L1 when MSHR occupancy is below half, otherwise into L2.

Accuracy-based throttling. Accuracy is measured in epochs of 256 prefetches issued to the lower level. At the end of each epoch, acc $= \frac{\Delta \text{ useful prefetches}}{\Delta \text{ prefetches issued}}$ over that epoch, and the degree for the next epoch is set as:

Epoch accuracy	Degree
$\geq 75\%$	4
50–75%	3
25–50%	2
$< 25\%$	1

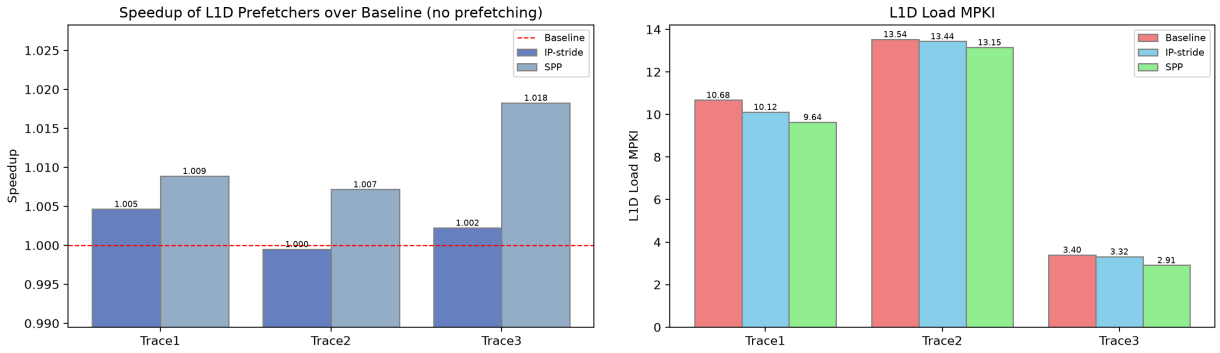
The initial degree is 2. A counter reset at the end of warmup is detected and simply restarts the epoch.

2.2 Results

Table 1: Task 1 results (25M warmup + 25M simulation instructions).

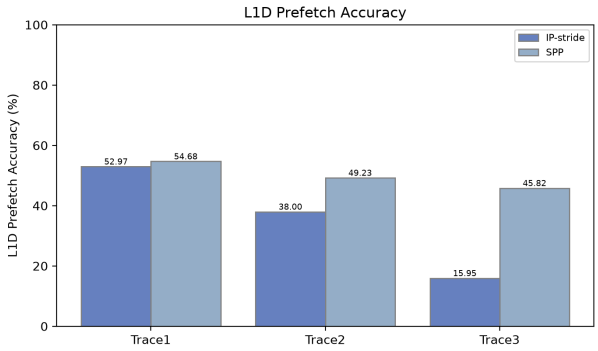
Trace	Config	IPC	Speedup	L1D Load MPKI	Accuracy (%)	Coverage (%)
Trace1	Baseline	0.2322	1.0000	10.679	–	–
	IP-stride	0.2333	1.0047	10.118	52.97	5.25
	SPP	0.2343	**1.0089**	**9.642**	**54.68**	**9.71**
Trace2	Baseline	0.2513	1.0000	13.536	–	–
	IP-stride	0.2512	0.9995	13.438	38.00	0.72
	SPP	0.2531	**1.0072**	**13.155**	**49.23**	**2.82**
Trace3	Baseline	0.3819	1.0000	3.403	–	–
	IP-stride	0.3827	1.0022	3.319	15.95	2.49
	SPP	0.3888	**1.0183**	**2.914**	**45.82**	**14.39**
Mean†	IP-stride	0.2820	1.0021	8.958	35.64	2.82
	SPP	0.2846	**1.0114**	**8.570**	**49.91**	**8.97**

†Geometric mean for IPC and speedup; arithmetic mean for MPKI, accuracy and coverage.

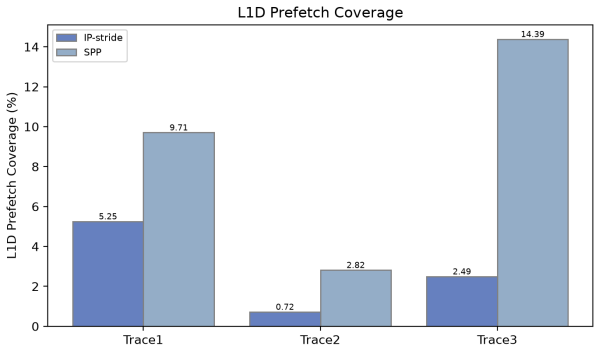


(a) Speedup

(b) L1D load MPKI



(c) Prefetch accuracy



(d) Prefetch coverage

Figure 1: IP-stride vs. SPP at the L1D.

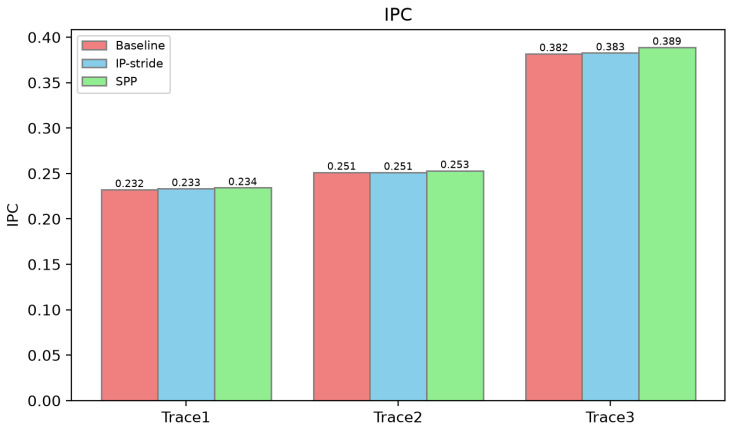


Figure 2: IPC of Baseline, IP-stride and SPP.

Observations.

- SPP outperforms IP-stride on every trace in all four metrics. Its geomean speedup is 1.0114 vs. 1.0021 for IP-stride, and it lowers average L1D load MPKI from 9.21 (baseline) to 8.57, vs. 8.96 for IP-stride.
- On **Trace3** SPP reaches 14.4% coverage vs. 2.5% for IP-stride, and accuracy rises from 16% to

46%. This large gap suggests that the stream contains irregular but repeating stride sequences, which IP-stride cannot capture (it prefetches wrongly when two strides coincidentally match) but the signature table can.

- On **Trace2** IP-stride is slightly *slower* than the baseline (0.9995). Its fixed degree 3 at 38% accuracy pollutes the L1D. SPP stays at 49% accuracy, because its throttle keeps the degree low when accuracy is below 50%, and gains 0.72%.
- Absolute speedups are small because all three traces are memory-bound (IPC $\approx$ 0.23–0.38) and coverage stays modest (3–14%). Most L1D misses are not predictable within a 4 KB page by an IP-local prefetcher.

2.3 Storage Overhead

Field widths are the minimum needed by the logic. Strides are in cache lines and never cross a 4 KB page (64 lines), so they lie in $[-63, +63]$ and need 7 bits (signed).

Table 2: IP table entry (64 entries).

Field	Bits	Purpose
`valid`	1	entry in use
`tag`	8	partial IP tag, `(ip / 64) & 0xFF`
`last_line`	58	last line address (64-bit address − 6 block-offset bits)
`stride`	7	uniform stride, $[-63, +63]$
`conf`	2	uniform confidence, 0–3
`sig`	9	stride-history signature
Total per entry	**85**	
Table total	$64 \times 85 =$ **5440** bits	

Table 3: Pattern table entry (512 entries, indexed by the 9-bit signature, so no tag).

Field	Bits	Purpose
`next`	7	predicted next stride, $[-63, +63]$
`conf`	2	confidence, 0–3
Total per entry	**9**	
Table total	$512 \times 9 =$ **4608** bits	

Table 4: Throttling state (global).

Field	Bits	Purpose
`degree`	2	current degree 1–4 (stored as degree−1)
epoch prefetch counter	9	prefetches issued in this epoch (up to 256, plus overshoot)
epoch useful counter	9	useful prefetches in this epoch
Total	**20**	

In the simulator the throttle stores 64-bit snapshots of ChampSim's global counters for convenience. In hardware, two 9-bit counters reset at each epoch are equivalent.

$$\text{Total} = 5440 + 4608 + 20 = \textbf{10068 bits} \approx \textbf{1258.5 bytes} \ (\approx 1.23 \text{ KB}).$$

The largest field is `last_line` (58 bits). It could be reduced to a 6-bit page offset plus a hashed page tag of about 16 bits, bringing the IP table to 2240 bits, but we report the design exactly as implemented.

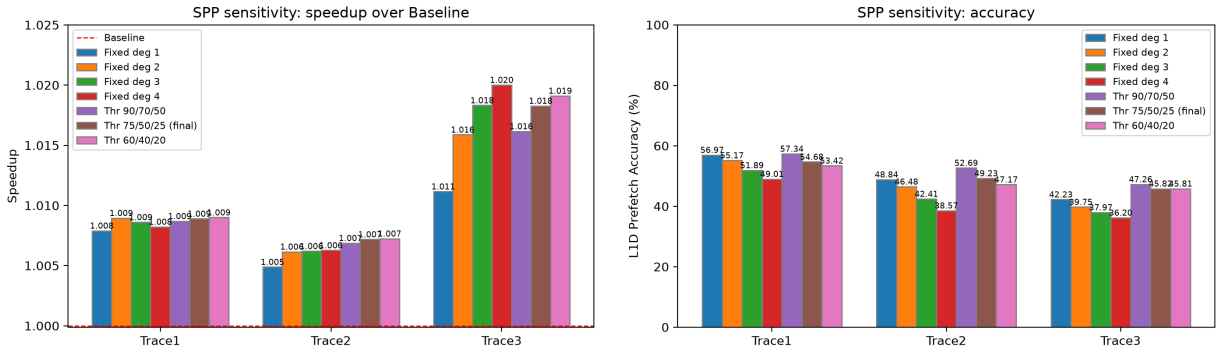
2.4 Choice of Accuracy Thresholds and Degrees

How the thresholds were decided. The goal of throttling is to use a large degree only when extra prefetches are likely to be useful. Accuracy is a fraction in $[0, 100]\%$ and we have four degree levels, so we started from an even quartile split (75/50/25). We then ran a sensitivity study comparing it against (i) fixed degrees 1–4 with throttling disabled, (ii) a conservative split 90/70/50 and (iii) an aggressive split 60/40/20. All other parameters were kept unchanged.

Table 5: Sensitivity of SPP to degree and throttling thresholds (speedup / accuracy% / coverage%).

Config	Trace1	Trace2	Trace3	Geomean SU	Prefetc
Fixed degree 1	1.0079 / 56.97 / 7.25	1.0049 / 48.84 / 1.97	1.0112 / 42.23 / 9.14	1.0080	79
Fixed degree 2	1.0090 / 55.17 / 9.13	1.0062 / 46.48 / 2.54	1.0159 / 39.75 / 13.50	1.0103	11
Fixed degree 3	1.0086 / 51.89 / 10.02	1.0062 / 42.41 / 2.67	1.0183 / 37.97 / 15.91	1.0110	14
Fixed degree 4	1.0082 / 49.01 / 10.53	1.0063 / 38.57 / 2.67	1.0200 / 36.20 / 17.79	1.0115	16
Thr 90/70/50	1.0087 / 57.34 / 8.69	1.0069 / 52.69 / 2.65	1.0162 / 47.26 / 11.83	1.0106	95
Thr 75/50/25 (final)	1.0089 / 54.68 / 9.71	1.0072 / 49.23 / 2.82	1.0183 / 45.82 / 14.39	**1.0114**	**11**
Thr 60/40/20	1.0090 / 53.42 / 10.03	1.0072 / 47.17 / 2.85	1.0191 / 45.81 / 15.33	1.0118	12

‡Total prefetches issued to the lower level, summed over the three traces (a proxy for extra memory traffic).



(a) Speedup (b) Accuracy

Figure 3: Sensitivity study of SPP degree and thresholds.

What the study shows.

- **No single fixed degree is best everywhere.** Trace1 peaks at degree 2 and gets *slower* at degrees 3 and 4. Trace3 keeps improving up to degree 4. Raising the degree always lowers

accuracy (e.g. Trace2 drops from 48.8% to 38.6%) while coverage saturates (Trace2 coverage is flat beyond degree 3). This is exactly the case for accuracy-based throttling.

- **Throttling gives most of the benefit of a large degree at a lower cost.** The final 75/50/25 setting reaches a geomean speedup of 1.0114, within 0.01% of fixed degree 4 (1.0115), but issues **29% fewer** prefetches (116.8K vs. 164.0K) at 8.6 points higher average accuracy (49.9% vs. 41.3%). It is also within 0.03% of the best result on Trace1 and Trace2.
- **Conservative thresholds (90/70/50)** have the best accuracy, but they rarely allow degree 3–4, so Trace3 loses coverage (11.8% vs. 14.4%) and speedup.
- **Aggressive thresholds (60/40/20)** are marginally faster (geomean 1.0118 vs. 1.0114) but issue 7.4% more prefetches for that 0.04% gain. Both their accuracy and their advantage shrink as more prefetches are sent.

We chose 75/50/25 as the balance point. It is essentially tied with the fastest configurations while keeping accuracy close to 50% and limiting cache pollution and memory traffic. This matters in a full system where the L2/LLC and DRAM bandwidth are shared, and it is the stated goal of the throttle.

Why these degrees for each accuracy range. The degree is chosen so that the *expected number of useful prefetches per trigger* grows with accuracy while the *expected number of useless prefetches* stays bounded at about 1–2:

- $\geq 75\% \rightarrow$ **degree 4.** At high accuracy, 4 prefetches yield about 3 useful lines and at most 1 useless one. Lookahead also improves timeliness: deeper prefetches are issued earlier relative to their demand (Trace3 late prefetches drop from 5262 at degree 2 to 3970 at degree 4). We cap the degree at 4 because a deeper signature walk compounds prediction errors (each lookahead step multiplies the probabilities), and coverage was already saturating at degree 3–4 in the study.
- 50–75% $\rightarrow$ **degree 3.** Each prefetch is still more likely to be useful than not, so lookahead is worth it, but one step less to limit wasted lines.
- 25–50% $\rightarrow$ **degree 2.** Fewer than half the prefetches help. Degree 2 still covers the immediate next access plus one lookahead, which helped all traces compared to degree 1 (fixed degree 2 beats fixed degree 1 on every trace).
- $< 25\% \rightarrow$ **degree 1.** Most prefetches are wasted. We keep a minimum degree of 1 rather than disabling the prefetcher, so it keeps issuing (and measuring) prefetches and can recover when the program enters a more predictable phase. A degree of 0 would leave no useful-prefetch feedback to raise accuracy again.

The epoch length of 256 prefetches is long enough for a statistically stable accuracy estimate (one useful prefetch changes it by only 0.4%) yet short enough to react to program phases within a few thousand accesses.

1 Task 2A: Instruction-Conscious L2 Replacement Policy (I-RRIP)

1.1 Goal and motivation

Server-style workloads have instruction footprints that exceed the L1I cache, so many instruction fetches are served by the L2. An L2 instruction miss stalls the front end (fetch and decode), which the out-of-order back end cannot hide, whereas many data misses can be overlapped with other work. Plain SRRIP treats both kinds of lines identically. In this task we extend SRRIP at the L2 so that instruction lines are favoured in all three parts of the policy (insertion, promotion and eviction), with the goal of reducing the L2 load MPKI of instruction lines.

1.2 Experimental setup

All results come from ChampSim with the configuration in Table 1. Every run simulates 25M warm-up instructions (caches and replacement state are trained, statistics discarded) followed by 25M measured instructions, as required by the assignment. No prefetcher is enabled at any level, and all levels except the L2 use LRU. We evaluate three L2 policies on all three provided traces (referred to as Trace1 to Trace3):

- **LRU (baseline):** the default policy with no prefetching. This is the reference for speedup.
- **SRRIP:** the provided `rrip.l2c_repl`, the policy we are improving on.
- **I-RRIP:** our instruction-conscious policy (`irrip.l2c_repl`).

Parameter	Value
Core	1 core, hashed perceptron branch predictor
L1D	64 sets × 12 ways (48 KiB), LRU
L2C	1024 sets × 8 ways (512 KiB), policy under test
LLC	2048 sets × 16 ways (2 MiB), LRU
DRAM	4096 MB, 1 channel, 6400 MT/s
Prefetchers	none at any level
Warm-up / measured	25M / 25M instructions

Table 1: Simulated configuration.

Metrics. IPC is ChampSim's cumulative IPC over the measured region. Speedup is the IPC of the policy divided by the IPC of LRU on the same trace. L2C load MPKI is the number of L2 demand-load misses per 1000 instructions, and ChampSim additionally reports it split into *instruction* and *data* load MPKI; the two add up to the total (for example I-RRIP on Trace2: $13.335 + 9.348 = 22.683$).

1.3 Design of I-RRIP

SRRIP keeps a 2-bit re-reference prediction value (RRPV, 0 to 3) per line. A new line is inserted at RRPV 2, a hit resets the line to 0, and the victim is a line with RRPV 3 (if none exists, all RRPVs in the set are incremented and the search repeats). I-RRIP changes all three steps using one extra bit per line, `is_instr`, plus the `isInstruction` flag that `cache.cc` now passes

to `update_replacement_state` (taken from the packet's `instruction` field; writebacks pass 0). Table 2 summarises the three changes.

Step	SRRIP	I-RRIP
Insertion	RRPV 2 for every line	Instruction line: RRPV 0. Data line: RRPV 2 (unchanged).
Promotion (hit)	RRPV $\rightarrow$ 0 for every line	Instruction line: RRPV $\rightarrow$ 0. Data line: RRPV decremented by one (not below 0).
Eviction	first line with RRPV 3	Among lines at RRPV 3, evict a data line first; evict an instruction line only if every line at RRPV 3 is an instruction line. If no line is at RRPV 3, age all lines and retry.

Table 2: SRRIP versus I-RRIP. Translation and other non-instruction lines are treated as data lines.

Insertion (modified). A line brought into the L2 by an instruction fetch means the L1I has just missed, so the front end is waiting on it, and the code footprint of server workloads usually fits in the L2 even when it does not fit in the L1I. We therefore insert instruction lines at RRPV 0, the highest-priority position, which gives them the longest possible time before they become eviction candidates. Data lines keep SRRIP's insertion at RRPV 2 so that streaming data is still filtered out quickly.

Promotion (modified). In SRRIP, a single hit makes a data line exactly as protected as an instruction line. Moving data lines down by only one step on a hit gives them a smaller reward, so heavily reused data cannot push all instruction lines toward eviction. Instruction lines still go directly to 0. The `is_instr` bit is set on insertion and also on any instruction hit, and is not cleared by data hits, so a line that has served instruction fetches stays protected.

Eviction (modified). Insertion and promotion only change when a line *becomes* a candidate. The victim choice is the final safeguard: when several lines have reached RRPV 3, evicting a data line first keeps instruction lines resident even after they have aged. Instruction lines are not pinned permanently. If every RRPV-3 line is an instruction line, one of them is evicted, so a set can never deadlock and a cold instruction line eventually leaves.

Why not a simpler or stronger variant. Changing insertion alone would let a data line promoted to 0 by one hit become as protected as instructions. Pinning instruction lines (never evicting them) would let stale code fill the set and starve data. The three-part bias above is a compromise that protects instructions strongly while still letting them age out. We did not run an ablation of each part individually, so the claim that each part contributes is supported by reasoning and by the eviction counters in Section 1.5, not by separate measurements.

Hardware cost. I-RRIP needs one extra bit per L2 line on top of SRRIP's 2-bit RRPV. With $1024 \times 8 = 8192$ lines this is 8192 bits (1 KiB), compared with 16384 bits (2 KiB) for the RRPVs themselves.

1.4 Results

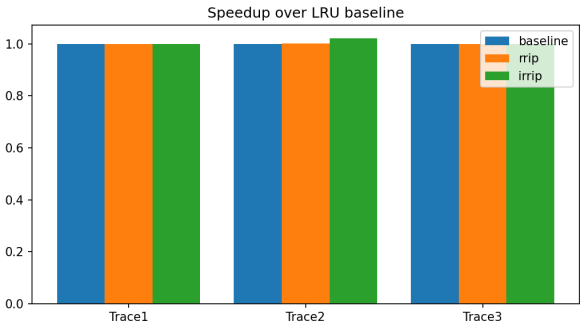
Table 3 lists the required metrics for every trace and policy (raw values are in the file `task2a_results.csv`). Table 4 adds hit rate and miss latency, and Figures 1–3 plot the same data. In the figures, the policy named "baseline" is LRU.

Trace	Policy	IPC	Speedup	L2C load MPKI	L2 instr load MPKI	L2 data load MPKI
Trace1	LRU	0.2323	1.0000	17.696	10.338	7.358
	SRRIP	0.2322	0.9995	17.639	10.562	7.077
	I-RRIP	0.2322	0.9992	17.762	9.856	7.907
Trace2	LRU	0.2512	1.0000	27.357	18.853	8.504
	SRRIP	0.2518	1.0026	25.746	17.781	7.965
	I-RRIP	0.2565	1.0211	22.683	13.335	9.348
Trace3	LRU	0.3821	1.0000	3.954	1.124	2.830
	SRRIP	0.3817	0.9991	4.039	1.197	2.842
	I-RRIP	0.3821	1.0001	3.879	0.996	2.883

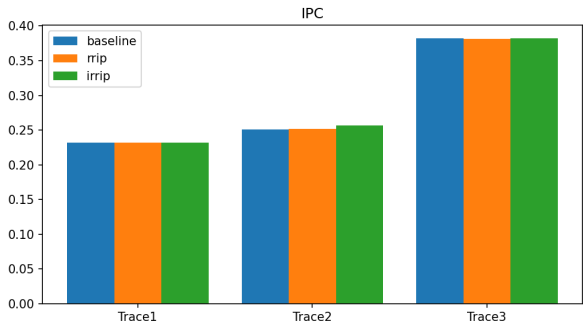
Table 3: IPC, speedup over LRU, and L2 demand-load MPKI (total, instruction, data) over the 25M measured instructions.

Trace	Policy	L2 load hit rate (%)	Avg. L2 miss latency (cycles)
Trace1	LRU	29.7	140.3
	SRRIP	29.9	140.3
	I-RRIP	29.4	134.1
Trace2	LRU	48.9	89.5
	SRRIP	51.9	93.1
	I-RRIP	57.7	93.9
Trace3	LRU	28.5	156.5
	SRRIP	26.9	151.5
	I-RRIP	29.8	154.1

Table 4: Supplementary metrics: L2 demand-load hit rate and the average L2 miss latency reported by ChampSim.

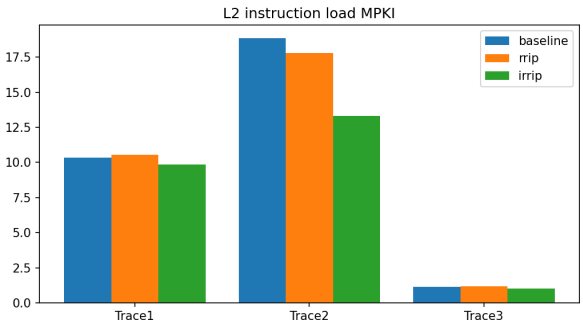


(a) Speedup over LRU.

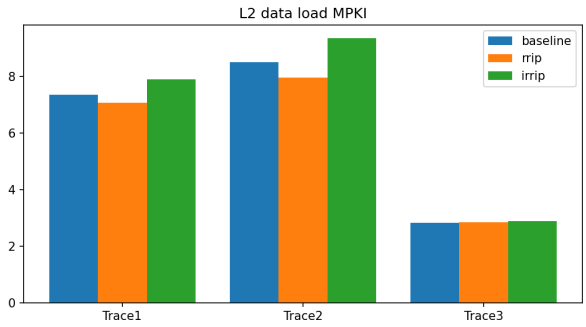


(b) IPC.

Figure 1: Performance of LRU ("baseline"), SRRIP and I-RRIP on the three traces.



(a) L2 instruction load MPKI.



(b) L2 data load MPKI.

Figure 2: L2 load MPKI split by line type.

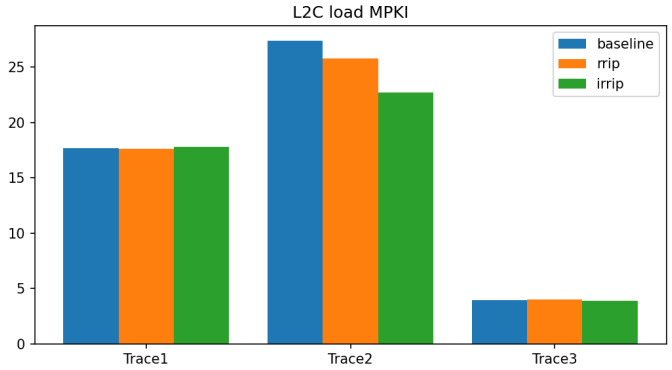


Figure 3: Total L2C load MPKI (instruction plus data).

1.5 Analysis

I-RRIP achieves its target. L2 instruction load MPKI is lower under I-RRIP than under both LRU and SRRIP on all three traces. Relative to SRRIP the reduction is 6.7% on Trace1 (10.56 → 9.86), 25.0% on Trace2 (17.78 → 13.33) and 16.8% on Trace3 (1.20 → 1.00).

Trace2: a clear gain. Trace2 has by far the highest instruction MPKI (18.9 under LRU), which is the situation I-RRIP is designed for. Instruction MPKI falls by 4.45 relative to SRRIP while data MPKI rises by only 1.38 (7.96 → 9.35), so total L2C load MPKI drops from 25.75 to 22.68 (11.9% lower than SRRIP, 17.1% lower than LRU). The L2 load hit rate rises from 48.9% (LRU) to 57.7%, and IPC improves by 2.11% over LRU and 1.85% over SRRIP. The IPC gain is much smaller than the MPKI reduction. Trace2's average L2 miss latency is only about 90 cycles, compared with about 135 to 155 on the other traces, which suggests that many of its L2 misses are served by the LLC rather than DRAM, so each avoided miss saves less time; we did not isolate this effect with a separate experiment.

Trace1: the trade-off. Instruction MPKI falls by 0.71 but data MPKI rises by 0.83 relative to SRRIP, so total MPKI is essentially unchanged (17.64 → 17.76) and so is IPC (speedup 0.9992 over LRU, 0.03% below SRRIP). The instruction gain is cancelled by extra data misses, i.e. blindly favouring instructions does not pay off when data misses cost as much as instruction misses.

Trace3: little headroom. Total L2 load MPKI is only about 4, so no replacement policy can change IPC noticeably (speedup 1.0001). I-RRIP still lowers both instruction MPKI and total MPKI (4.04 → 3.88 versus SRRIP).

SRRIP versus LRU. SRRIP gives slightly *higher* instruction MPKI than LRU on Trace1 (10.56 vs. 10.34) and Trace3 (1.20 vs. 1.12). SRRIP's scan resistance helps data, but it gives instruction lines no preference, so the instruction gains seen in I-RRIP come from the instruction-aware changes and not from RRIP itself.

Evidence from eviction counters. ChampSim prints counters of the form "L2C X Evicting Y". Reading X as the type of the incoming line and Y as the type of the resident line it displaces, Table 5 sums, for each trace, how many instruction lines and data lines were evicted (over all incoming types). We use these counters only to compare policies on the same trace. I-RRIP evicts fewer instruction lines on every trace and more data lines, which is the intended shift and matches the MPKI split. On Trace2 the number of instruction lines evicted by incoming instruction lines alone falls from 284,190 to 173,076 (39% lower), indicating that SRRIP lets instruction lines evict each other heavily on this trace.

	Instruction lines evicted			Data lines evicted		
Trace	SRRIP	I-RRIP	Δ	SRRIP	I-RRIP	Δ
Trace1	261,441	236,236	−9.6%	273,029	308,367	+12.9%
Trace2	431,648	300,977	−30.3%	289,934	358,042	+23.5%
Trace3	31,970	27,270	−14.7%	105,044	106,190	+1.1%

Table 5: Resident instruction and data lines evicted from the L2, summed over all incoming line types, from ChampSim's "L2C X Evicting Y" counters.

1.6 Limitations and link to Task 2B

The static policy helps when instruction misses dominate (Trace2), is neutral on a trace with little L2 pressure (Trace3), and loses its gain to extra data misses when the two kinds of misses are comparable (Trace1). The data MPKI increase on every trace is the price of always favouring

instructions, and it motivates the dynamic, set-dueling version in Task 2B, which can fall back to plain SRRIP in phases where prioritising instructions does not help. Our conclusions rest on three traces and a single 25M-instruction window per trace, and we did not run per-component ablations, so they should not be generalised beyond this setup.

1.7 Reproducing the results

```
./build_champsim.sh no lru   baseline
./build_champsim.sh no rrip  rrip
./build_champsim.sh no irrip irrip
bin/<binary> -warmup_instructions 25000000 \
    -simulation_instructions 25000000 \
    -traces /traces/<Trace>.gz > results/<policy>_<Trace>.txt
python3 parse.py   # writes task2a_results.csv and plot_*.png
```

2 Task 2B: Dynamic I-RRIP

2.1 Goal and motivation

Task 2A showed that statically favouring instruction lines at the L2 helps when instruction misses dominate (Trace2) but trades instruction misses for data misses everywhere, because instruction lines are favoured even when the program is not limited by its front end. Task 2B makes the choice dynamic: a few *leader* sets always run plain SRRIP and measure how much of the L2 traffic belongs to instruction lines; the remaining *follower* sets prioritise instructions only while that share is above a threshold, and run SRRIP otherwise (set dueling, as discussed in class).

Because the fallback to SRRIP protects data-heavy phases, the instruction mode itself can be made considerably more aggressive than the static Task 2A policy. This turned out to be the main source of the improvement over Task 2A (Sections 2.6 and 2.8).

2.2 Experimental setup

The setup is identical to Task 2A: ChampSim, 1 core, hashed perceptron branch predictor, 48 KiB L1D, 512 KiB 8-way L2 (policy under test), 2 MiB 16-way LLC, no prefetcher at any level, 25M warm-up followed by 25M measured instructions, on the three provided traces. All binaries were built and run with `gcc 7.5`. We evaluate four L2 policies:

- **LRU (baseline):** the reference for speedup.
- **SRRIP:** `rrip.l2c_repl`.
- **I-RRIP:** the Task 2A policy `irrip.l2c_repl`. To make the comparison like for like, we re-ran it in the same environment as Task 2B. Our numbers agree with the Task 2A report to within about 0.3%.
- **Dynamic I-RRIP:** this task, `dyn_irrip.l2c_repl`, at five threshold values.

Metrics are the same as in Task 2A. IPC is the cumulative IPC over the measured region. Speedup is IPC divided by the IPC of LRU on the same trace. L2C load MPKI is reported in total and split into instruction and data load MPKI. Averages over traces use the geometric mean for IPC and speedup and the arithmetic mean for MPKI.

2.3 Design of Dynamic I-RRIP

Follower modes. Table 1 lists the two modes a follower set can be in. SRRIP mode is exactly `rrip.l2c_repl`. The instruction mode keeps Task 2A's insertion and promotion, but has a *strict* eviction rule. While the set holds any data line, the set is aged until a data line reaches RRPV 3, and that data line is evicted. An instruction line is evicted only from an all-instruction set, and then by plain SRRIP. Task 2A's I-RRIP, by contrast, evicts an instruction line whenever no data line happens to be at RRPV 3, even if data lines with lower RRPV are present.

Step	SRRIP mode	Instruction mode (strict I-RRIP)
Insertion	RRPV 2 for every line	Instruction line: RRPV 0. Data line: RRPV 2.
Promotion	RRPV $\rightarrow$ 0 for every line	Instruction line: RRPV $\rightarrow$ 0. Data line: RRPV -1.
Eviction	first line with RRPV 3	If the set has a data line: age the set until a data line reaches RRPV 3, evict it. Otherwise: SRRIP.

Table 1: The two follower modes. Translation lines are treated as data lines, as in Task 2A.

The per-line `is_instr` bit from Task 2A is kept up to date in every set and every mode, so a switch to instruction mode immediately knows which resident lines hold code. Switching does not touch cache contents; lines adapt through normal aging.

The strict rule is safe here and not in Task 2A for a specific reason. Pinning code is harmful precisely when the program is data-bound. In that case the leader sets measure a low instruction share and the followers fall back to SRRIP, where code gets no protection at all.

Leader sets. 16 of the 1024 sets (1.6%) are leaders and always run SRRIP. In every group of 64 consecutive sets, the leader is the set whose offset equals the group number (`set % 64 == (set / 64) % 64`: sets 0, 65, 130, …, 975), so the samples are spread evenly over the index space. The leaders run SRRIP and not I-RRIP because the measurement must show the workload's real instruction pressure. Under instruction priority, code would rarely miss and hide how much the program needs it, creating a feedback loop.

Decision metric. In the leader sets we count demand accesses (LOAD, RFO, translations; writebacks and prefetches are excluded) and misses. Each hit is one access, and each fill is one access plus one miss. Following the hint in the assignment,

$$r = \frac{\text{instr. accesses} + \text{instr. misses}}{\text{total accesses} + \text{total misses}}, \qquad \text{followers} = \begin{cases} \text{instruction mode} & r > T \\ \text{SRRIP} & \text{otherwise} \end{cases}$$

so misses are counted twice and weigh more than hits. We use $T = 0.05$, chosen by the sensitivity study in Section 2.5.

Epochs and decay. The decision is re-evaluated every 512 leader-set demand accesses. After each evaluation all four counters are halved. The metric is thus an exponentially decaying average (weights $1/2, 1/4, \ldots$ for older epochs). It reacts to a phase change within one or two epochs, but a single noisy epoch cannot flip the mode. With purely cumulative counters, the decision would settle on the program's average and stop following phases.

Correctness check. With the threshold forced to 2 (never instruction mode), the policy is cycle-for-cycle identical to `rrip.l2c_repl` on Trace1 (107 659 661 cycles in both runs). This shows that the leader/follower bookkeeping has no side effects.

Hardware cost. On top of I-RRIP (2-bit RRPV and 1 `is_instr` bit per line, 24 576 bits), Dynamic I-RRIP adds four 10-bit leader counters (halving every 512 accesses keeps them below 1024), a 9-bit epoch counter and a mode bit: **50 bits**. Leader identification is a comparator on set-index bits. With $T = 0.05$ the test $r > T$ is $20(i_a + i_m) > (t_a + t_m)$, where $20x = (x \ll 4) + (x \ll 2)$: two shifts, an add and a compare, with no divider.

2.4 Number of leader sets

Every leader is a set that never receives instruction priority, so leaders have a direct performance cost. Their benefit, a stable estimate of r, saturates quickly because r is a property of the whole workload. Table 2 varies only the number of leaders (strict instruction mode, $T = 0.10$, epoch 1024).

Leader sets	% of cache	Trace1	Trace2	Trace3	Geomean speedup
64	6.3%	1.0086	1.0291	1.0012	1.0129
32	3.1%	1.0099	1.0294	1.0055	1.0149
16	**1.6%**	1.0099	1.0352	1.0056	**1.0168**

Table 2: Speedup over LRU for different numbers of leader sets.

Fewer leaders are strictly better on these traces. The gain is largest on the front-end-bound Trace2, where 16 more protected sets lower the L2 instruction MPKI from 9.80 to 9.25. We stop at 16 because the light-traffic Trace3 already produces only six decisions in 25M instructions; with 8 leaders the estimate would rest on very few accesses and become noisy. To keep the number of accesses *per leader set* per decision the same as with 32 leaders, the final design halves the epoch from 1024 to 512 accesses.

2.5 Sensitivity study of the threshold

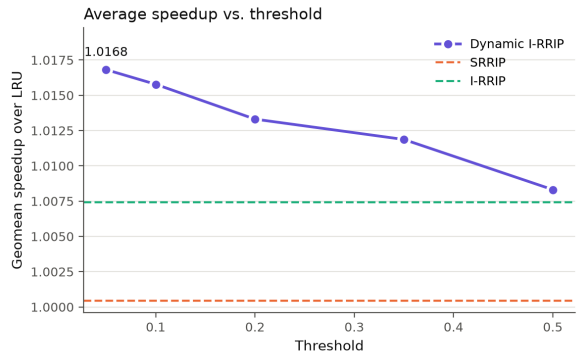
We swept $T \in \{0.05, 0.10, 0.20, 0.35, 0.50\}$. The threshold is read from the environment variable `DYN_IRRIP_THRESHOLD`, so one binary covers every value. The values were placed around the ratios the leaders actually measure (Table 3), so that each trace's ratio falls between two of them.

Trace	Measured r	L1I load MPKI	Character
Trace1	≈0.42–0.46	14.5	instruction and data misses of similar size
Trace2	≈0.65	40.0	front-end bound, large code footprint
Trace3	≈0.14–0.21	2.1	small code footprint, light L2 traffic

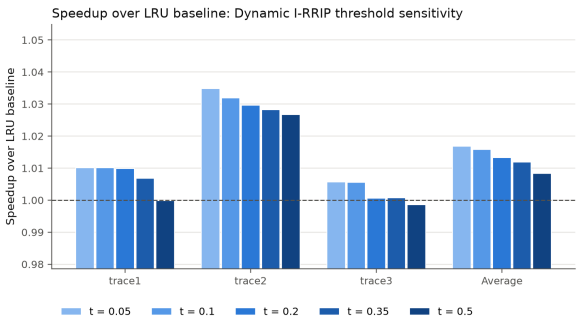
Table 3: Instruction share r measured by the SRRIP leader sets (end-of-run values over the five runs; Trace3 fluctuates).

	$T = 0.05$	$T = 0.10$	$T = 0.20$	$T = 0.35$	$T = 0.50$
Speedup over LRU					
Trace1	1.0101	1.0101	1.0099	1.0068	0.9999
Trace2	1.0349	1.0319	1.0297	1.0283	1.0267
Trace3	1.0057	1.0056	1.0006	1.0007	0.9986
Geomean	**1.0168**	1.0158	1.0133	1.0119	1.0083
L2 instruction / data load MPKI					
Trace1	9.07 / 9.97	9.07 / 9.97	9.07 / 9.97	9.18 / 9.86	10.56 / 7.06
Trace2	9.26 / 12.11	9.53 / 11.90	9.69 / 11.82	9.96 / 11.74	10.26 / 11.70
Trace3	0.73 / 3.27	0.73 / 3.27	0.94 / 3.12	0.99 / 3.05	1.05 / 3.00
Fraction of measured epochs in instruction mode / mode switches					
Trace1	1.00 / 0	1.00 / 0	1.00 / 0	0.96 / 2	0.00 / 0
Trace2	1.00 / 0	0.98 / 2	0.96 / 2	0.91 / 2	0.89 / 2
Trace3	1.00 / 0	1.00 / 0	0.50 / 1	0.33 / 1	0.33 / 1

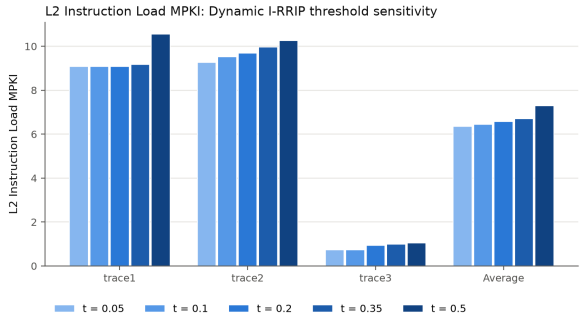
Table 4: Threshold sensitivity of Dynamic I-RRIP (16 leaders, epoch 512). The measured region contains 28, 47 and 6 epochs for Trace1–3.



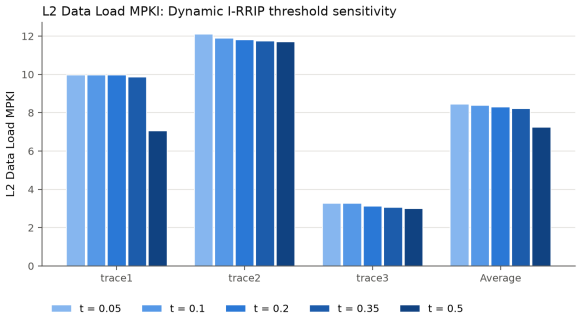
(a) Geomean speedup vs. threshold.



(b) Per-trace speedup.



(c) L2 instruction load MPKI.



(d) L2 data load MPKI.

Figure 1: Threshold sensitivity. Panel (a) also shows SRRIP and I-RRIP for reference.

The decision follows the measured ratio. Trace1 ($r \approx 0.43$) leaves instruction mode at $T = 0.50$ and then behaves exactly like SRRIP (speedup 0.9999, data MPKI back to 7.06). Trace3 ($r \approx 0.14$–0.21) spends half or more of its epochs in SRRIP from $T = 0.20$ upwards. Trace2 ($r \approx 0.65$) stays in instruction mode almost all the time at every threshold.

Speedup decreases monotonically with T. On all three traces, an epoch spent in SRRIP instead of instruction mode costs IPC. At $T = 0.20$ Trace3 loses most of its gain (1.0057 → 1.0006); at $T = 0.50$ Trace1 loses all of it (1.0101 → 0.9999). Trace2 is also sensitive despite its high average ratio: its decayed ratio occasionally dips below the higher thresholds, and those SRRIP epochs let code be evicted (instruction MPKI 9.26 → 10.26 from $T = 0.05$ to 0.50).

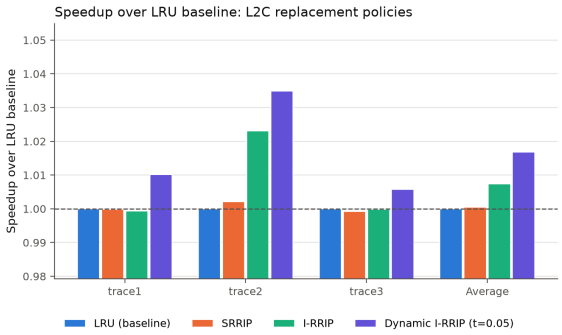
Choice of $T = 0.05$. It is the best value on every trace. Its meaning is "protect instructions unless the L2 sees essentially no instruction traffic". That still covers the case the assignment warns about, a data-dominated phase with no code pressure, while not giving up instruction priority on workloads with a moderate code share such as Trace3. Thresholds up to 0.10 are within 0.1% of the best, so the choice is not fragile.

2.6 Results

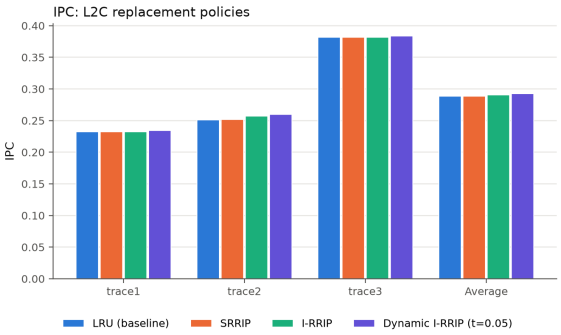
Table 5 reports the required metrics with the final configuration ($T = 0.05$), and Figures 2–3 plot them. Raw values are in `results/task2b/task2b_results.csv`, and all logs are in `results/task2b/`.

Trace	Policy	IPC	Speedup	L2C load MPKI	L2 instr load MPKI	L2 data load MPKI
Trace1	LRU	0.2322	1.0000	17.703	10.368	7.334
	SRRIP	0.2322	0.9999	17.619	10.563	7.056
	I-RRIP	0.2321	0.9994	17.781	9.871	7.910
	Dyn I-RRIP	**0.2346**	**1.0101**	19.042	**9.072**	9.970
Trace2	LRU	0.2513	1.0000	27.560	18.892	8.668
	SRRIP	0.2518	1.0022	26.012	17.980	8.032
	I-RRIP	0.2571	1.0231	22.292	12.834	9.458
	Dyn I-RRIP	**0.2601**	**1.0349**	**21.379**	**9.264**	12.115
Trace3	LRU	0.3819	1.0000	3.960	1.124	2.837
	SRRIP	0.3816	0.9993	4.037	1.190	2.846
	I-RRIP	0.3818	0.9999	3.891	0.998	2.893
	Dyn I-RRIP	**0.3840**	**1.0057**	3.994	**0.727**	3.267
Average	LRU	0.2814	1.0000	16.408	10.128	6.280
	SRRIP	0.2815	1.0004	15.889	9.911	5.978
	I-RRIP	0.2835	1.0074	14.655	7.901	6.753
	Dyn I-RRIP	**0.2861**	**1.0168**	14.805	**6.354**	8.451

Table 5: IPC, speedup over LRU, and L2 demand-load MPKI (total, instruction, data) over the 25M measured instructions. Dynamic I-RRIP uses $T = 0.05$.

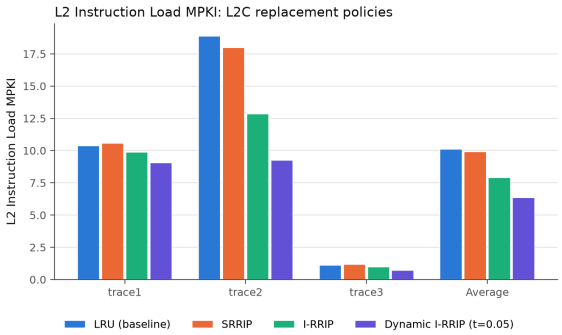


(a) Speedup over LRU.

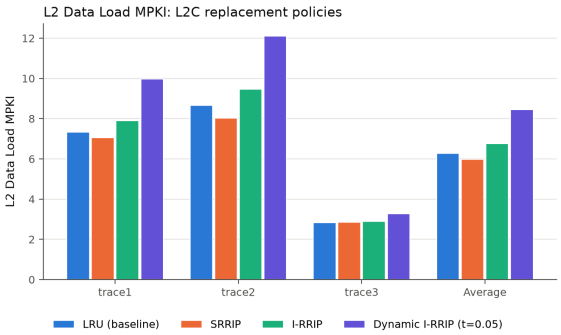


(b) IPC.

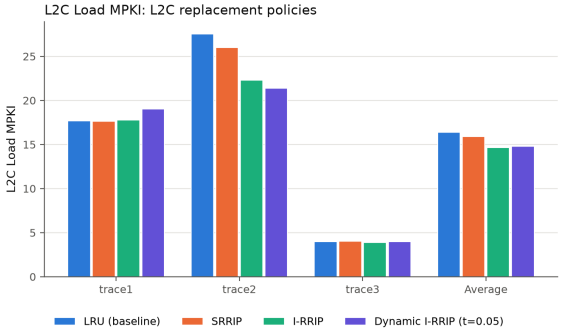
Figure 2: Performance of LRU, SRRIP, I-RRIP and Dynamic I-RRIP ($T = 0.05$).



(a) L2 instruction load MPKI.



(b) L2 data load MPKI.



(c) Total L2C load MPKI.

Figure 3: L2 load MPKI, in total and split by line type.

Trace	Policy	L2 load hit rate (%)	Avg. L2 miss latency (cycles)	LLC load misses
Trace1	LRU	29.6	140.5	307 044
	SRRIP	30.0	140.5	306 378
	I-RRIP	29.3	134.1	307 297
	Dyn I-RRIP	24.3	107.1	292 496
Trace2	LRU	48.5	88.9	204 533
	SRRIP	51.4	92.4	207 319
	I-RRIP	58.4	94.6	209 042
	Dyn I-RRIP	60.1	82.8	206 936
Trace3	LRU	28.4	156.9	78 933
	SRRIP	27.0	152.1	79 013
	I-RRIP	29.6	154.5	79 274
	Dyn I-RRIP	27.8	109.3	74 944

Table 6: Supplementary metrics: L2 demand-load hit rate, the average L2 miss latency reported by ChampSim, and LLC demand-load misses (i.e. loads that went to DRAM).

	Instruction lines evicted				Data lines evicted			
Trace	SRRIP	I-RRIP	Dyn	Δ vs I-RRIP	SRRIP	I-RRIP	Dyn	Δ vs I-RRIP
Trace1	261 682	236 891	197 782	−16.5%	272 226	308 096	397 319	+29.0%
Trace2	436 391	289 854	200 304	−30.9%	291 849	359 290	457 032	+27.2%
Trace3	31 843	27 320	15 786	−42.2%	105 047	106 452	129 104	+21.3%

Table 7: Resident instruction and data lines evicted from the L2, summed over all incoming line types, from ChampSim's "L2C X Evicting Y" counters (same method as Table 5 of Task 2A).

2.7 Analysis

Dynamic I-RRIP is the fastest policy on every trace. The geometric-mean speedup is 1.0168 over LRU: +1.64% over SRRIP and +0.93% over the Task 2A I-RRIP. It wins on every individual trace: +1.1% over I-RRIP on Trace1, +1.2% on Trace2 and +0.6% on Trace3. Trace1 and Trace3, where static I-RRIP was neutral, now gain about 1% and 0.6% over LRU.

The target metric, instruction MPKI, drops sharply. Relative to I-RRIP, L2 instruction load MPKI falls by 8.1% on Trace1 (9.87 → 9.07), 27.8% on Trace2 (12.83 → 9.26) and 27.2% on Trace3 (1.00 → 0.73). On average it falls by 20% relative to I-RRIP and 36% relative to SRRIP. Table 7 confirms the mechanism: instruction-line evictions fall by 16–42% relative to I-RRIP.

Data MPKI rises; this is the deliberate trade-off. Data-line evictions rise by 21–29%, and data load MPKI rises on every trace (mean 6.75 → 8.45 vs I-RRIP). Total L2C load MPKI is lower than I-RRIP only on Trace2; on Trace1 it is even higher than SRRIP (19.04 vs 17.62). Yet Trace1 runs 1% faster. Two effects explain why trading data misses for instruction misses pays off:

1. An L2 instruction miss stalls fetch and decode, and the out-of-order back end cannot hide it. Data misses can overlap with other independent work.

2. The extra data misses are cheap: they are mostly served by the LLC. The average L2 miss latency *drops* from 134.1 to 107.1 cycles on Trace1, 94.6 to 82.8 on Trace2 and 154.5 to 109.3 on Trace3 (Table 6). LLC load misses, i.e. trips to DRAM, are lower than under both SRRIP and I-RRIP on every trace (e.g. Trace1: 292 496 vs 306 378 and 307 297). The data lines that the strict mode evicts early are typically still in the 2 MiB LLC when they are needed again. A plausible reason why DRAM traffic even falls is that code held in the L2 no longer needs to compete for LLC capacity. We did not isolate this effect with a separate experiment.

Lower hit rate does not mean slower. On Trace1 the L2 load hit rate falls from 29.3% (I-RRIP) to 24.3%, but IPC rises by 1.1%. Hit rate counts every line equally, whereas the performance cost of a miss depends on its type and on where it is served from. This is why the policy optimises for instruction misses rather than total misses.

2.8 Design exploration

Our first version was a direct reading of the assignment: switching between SRRIP and the Task 2A I-RRIP, with 32 leaders, an epoch of 1024 and $T = 0.25$. It reached only 1.0063, slightly *below* static I-RRIP (1.0074). None of the three traces has a phase where prioritising instructions clearly hurts IPC, so switching saved almost nothing, while the SRRIP leaders cost some protection. We then varied one idea at a time (Table 8).

Variant (all with $T = 0.25$ unless stated)	Trace1	Trace2	Trace3	Geomean
First design: SRRIP $\leftrightarrow$ Task 2A I-RRIP, 32 leaders	0.9995	1.0204	0.9992	1.0063
+ data hits promoted to RRPV 0	0.9996	1.0177	0.9992	1.0055
16 leaders instead of 32	0.9995	1.0201	0.9999	1.0064
Strict mode only when $r > 0.55$	0.9995	1.0283	0.9988	1.0088
Strict mode only when $r > 0.35$	1.0096	1.0287	1.0008	1.0130
Strict mode whenever instruction mode is on	1.0099	1.0297	1.0006	1.0133
with $T = 0.10$	1.0099	1.0294	1.0055	1.0149
with $T = 0.10$, 16 leaders (final design)	1.0099	1.0352	1.0056	**1.0168**

Table 8: Design iterations, as speedup over LRU. "Strict mode" is the eviction rule of Table 1. Raw numbers are in `results/task2b/design_exploration/variants_summary.txt`.

Promoting data hits to RRPV 0 made data lines stay longer and push out code, so it was dropped. The strict eviction rule was the decisive change: it improves every trace, at every ratio where we enabled it. A three-level policy (SRRIP / relaxed I-RRIP / strict I-RRIP) therefore has no benefit on these traces, and the final policy has just two modes. The final row in Table 8 was re-run with the epoch halved to 512 for the 16-leader design; that is the configuration evaluated in Sections 2.5–2.6.

2.9 Limitations

The conclusions rest on three traces and a single 25M-instruction window per trace. The strict mode is safe only because of the dynamic fallback, and on these traces the fallback is rarely triggered at $T = 0.05$. A workload with long data-only phases would exercise it more. The threshold, leader count and epoch were tuned on the same three traces they are evaluated on, so the gains may be smaller on unseen workloads. The LLC explanation in the Analysis

is supported by the latency and DRAM-miss counters, but was not isolated with a separate experiment.

2.10 Reproducing the results

```
./build_champsim.sh no lru       baseline
./build_champsim.sh no rrip      rrip
./build_champsim.sh no irrip     irrip
./build_champsim.sh no dyn_irrip dyn_irrip
# the provided traces are gzip files without an extension
mkdir -p traces
for i in 1 2 3; do ln -sf ../Trace$i-00$i traces/trace$i.champsimtrace.gz; done
task2b/run_task2b.sh traces 8      # 3 traces x {LRU, SRRIP, I-RRIP, Dyn @ 5 T}
python3 task2b/parse_results.py    # results/task2b/task2b_tables.md, *.csv
python3 task2b/plot_results.py --dyn dyn_irrip_t0.05   # results/task2b/plots/
# single run with a chosen threshold (default 0.05)
DYN_IRRIP_THRESHOLD=0.05 bin/<dyn_irrip binary> -warmup_instructions 25000000 \
    -simulation_instructions 25000000 -traces traces/trace1.champsimtrace.gz
```

3 Task 3: Incorporating SPP with Dynamic I-RRIP

3.1 Goal

Task 3 runs the Stride Pattern Prefetcher (SPP) from Task 1B at the L1D together with the Dynamic I-RRIP L2 policy from Task 2B. The L2 replacement policy is then made *aware* of the lines that SPP brings in, which it can recognise by the packet type `PREFETCH`. The target is a higher IPC than both standalone SPP and Dynamic I-RRIP.

The two mechanisms attack different misses. SPP is a data prefetcher: it removes L1D data misses and does nothing for instructions. Dynamic I-RRIP removes L2 instruction misses and pays for it with extra L2 data misses. The prefetch-aware part decides how the L2 should treat the extra, speculative lines that SPP adds to the L2 traffic.

3.2 Experimental setup

The setup is the same as in Tasks 1B and 2B: ChampSim, 1 core, hashed perceptron branch predictor, 48 KiB 12-way L1D, 512 KiB 8-way L2, 2 MiB 16-way LLC, 25M warm-up followed by 25M measured instructions on the three provided traces. All binaries were built and run with `gcc 7.5`. The only prefetcher is the L1D prefetcher. We evaluate five configurations:

- **Baseline:** no prefetcher, LRU at the L2. This is the reference for speedup and coverage, as in Task 1B.
- **Dynamic I-RRIP:** no prefetcher, the Task 2B policy at the L2 ($T = 0.05$).
- **SPP + LRU:** standalone SPP (`prefetcher/spp.l1d_pref`, unchanged from Task 1B).
- **SPP + Dyn I-RRIP (unaware):** SPP with the Task 2B policy, which treats prefetched lines like any other data line. This is the same binary as the next configuration, run with `DYN_IRRIP_PF_AWARE=0`.
- **SPP + PF-aware Dyn I-RRIP:** this task, `replacement/dyn_irrip.l2c_repl`.

All five configurations were re-run on the current code base (with the Task 2 changes merged), so that they can be compared like for like. The baseline IPCs are identical to those of Tasks 1B and 2B. Our SPP + LRU speedups (1.0087, 1.0065, 1.0160) are within 0.25% of the Task 1B report.

Metrics. IPC is the cumulative IPC of the measured region, and Speedup = $\text{IPC}/\text{IPC}_{\text{baseline}}$. Prefetch accuracy is ChampSim's L1D statistic, *useful load prefetches / prefetches issued to the lower level*, as in Task 1B. Coverage is

$$\text{Coverage} = \frac{\text{L1D load misses}_{\text{baseline}} - \text{L1D load misses}}{\text{L1D load misses}_{\text{baseline}}}.$$

L1D and L2C load MPKI are taken from ChampSim's statistics, and the L2 load MPKI is split into instruction and data loads as in Task 2. Averages use the geometric mean for IPC and speedup and the arithmetic mean for everything else.

3.3 How SPP's prefetches reach the L2

Before designing the policy, we measured what the L2 actually sees from SPP (Table 1).

Trace	L2 policy	SPP prefetch requests	Sent to the L2	L2 prefetch hits (rate)	L2 prefetch fills	Prefetched lines demand-hit in L2	LLC load misses
Trace1	LRU	777 706	64 650	17 493 (27.6%)	45 833	–	293 113
	Dyn (unaware)	778 996	64 781	5 026 (7.9%)	58 397	1 013 (1.7%)	277 907
	Dyn (PF-aware)	779 929	64 706	5 006 (7.9%)	58 364	981 (1.7%)	277 422
Trace2	LRU	522 717	27 901	7 331 (26.4%)	20 391	–	196 869
	Dyn (unaware)	516 212	27 811	2 461 (8.9%)	25 137	490 (1.9%)	199 386
	Dyn (PF-aware)	514 884	27 630	2 437 (8.9%)	24 951	529 (2.1%)	199 287
Trace3	LRU	827 440	34 024	2 890 (8.6%)	30 583	–	63 285
	Dyn (unaware)	826 212	33 975	1 017 (3.0%)	32 426	382 (1.2%)	58 942
	Dyn (PF-aware)	825 041	33 887	1 003 (3.0%)	32 310	388 (1.2%)	58 886

Table 1: SPP's traffic at the L2 over the 25M measured instructions. "Sent to the L2" is ChampSim's *prefetches issued to lower level*. "Prefetched lines demand-hit in L2" counts prefetch fills that later served a demand access in the L2, out of all prefetch fills. It is measured by the policy's statistics, which LRU does not have.

Three observations shaped the design:

1. **The L2 sees little prefetch traffic.** More than 90% of SPP's requests never leave the L1D, because the line is already there or the request is merged. Only 1.1–2.6 prefetches per kilo-instruction reach the L2, compared with 24, 53 and 4.9 L2 demand loads per kilo-instruction on Trace1–3. Any prefetch-specific rule can therefore move IPC only a little.

2. **The L2 copy of a prefetched line is rarely used by a demand access.** Almost all prefetches are `FILL_L1`: 27 274 of 27 901 on Trace2 have the same fill and origin level. The line is placed in the L1D as well, and the L1D serves the demand access. Only 1.2–2.1% of prefetch fills ever get a demand hit in the L2. The rest are evicted unused, or are still resident at the end of the run.

3. **Dynamic I-RRIP lowers the L2 hit rate of prefetch requests** from 26–28% (LRU) to 8–9% on Trace1 and Trace2. Its strict instruction mode evicts data first, so the lines that SPP asks for again are more often gone. Each such miss becomes an LLC access, which makes the prefetch later.

3.4 Design: handling the prefetch packet

The prefetch awareness consists of three rules in `l2c_update_replacement_state`, all keyed on `type == PREFETCH`. Every other packet type is handled exactly as in Task 2B. With no prefetcher, the policy is therefore identical to Task 2B: the no-prefetch runs reproduce the Task 2B results cycle for cycle (Trace1: 106 568 235 cycles in both runs; likewise on Trace2 and Trace3).

Event	Task 2B (unaware)	Task 3 (prefetch-aware)
Prefetch request in a leader set	not counted	not counted (unchanged)
Prefetch fill (miss), any set, any mode	RRPV 2 (as demand data)	**RRPV 1**
Prefetch request hits in the L2	normal promotion	normal promotion (kept)
Demand hit on a prefetched line	normal promotion	normal promotion
Eviction	strict data-first / SRRIP	unchanged; prefetched lines are data

Table 2: Treatment of prefetched lines. The RRPV ordering in instruction mode becomes *code (0) < prefetched data (1) < demand data (2).* In SRRIP mode and in the leader sets, prefetched lines are inserted one step further from eviction than demand fills. In both, lower RRPV means kept longer.

Rule 1: prefetches never enter the set-dueling decision. The leader-set counters count only demand accesses (loads, RFOs, translations). Prefetches are speculative, they are always data, and their number depends on how aggressive SPP currently is, not on what the core is waiting for. Counting them would lower the instruction share r whenever SPP is aggressive, and could switch off instruction priority for the wrong reason. Task 2B already filtered prefetches out of the counters, but without a prefetcher the rule never applied; with SPP it is what keeps the mode decision correct. With SPP running, the leaders still measure $r \approx 0.42$–0.45, 0.66 and 0.17 on Trace1–3, similar to the values without a prefetcher in Task 2B.

Rule 2: a prefetch fill is inserted at RRPV 1. This value is a compromise between two facts from Section 3.3.

- *Do not pin prefetched lines.* Their L2 copy rarely serves a demand access, because the L1D holds its own copy. When we shielded unused prefetched lines from eviction while other data was present, the speedup fell from 1.0301 to 1.0235 (Table 6), and LLC load misses rose (Trace1: 284 727 instead of 277 422). The extra resident prefetches pushed out demand data that was about to be reused.
- *Do not throw them away either.* Inserting them at RRPV 3 ("dead on arrival") lowers the speedup to 1.0284 and raises LLC load misses on every trace. For example, on Trace1 it is 279 727 compared with 277 422. A prefetched line is still a predicted future demand access: SPP's L1D accuracy is 38–53%. The L1D (48 KiB) may drop the line before it is used, and SPP regularly requests the same lines again (the L2 prefetch hits in Table 1). About 2% of the prefetches have a different fill level (`FILL_L2`, issued when the L1D MSHR is more than half full), and for those the L2 copy is the only copy. The replacement interface receives only the packet type, so these cannot be told apart from `FILL_L1` prefetches, and a single mild level of protection is used for all.

RRPV 1 places a prefetched line just above demand-fetched data but below code. A demand-fetched data line has already delivered its first use to the L1D, whereas a prefetched line has not been used yet. It keeps the strict instruction priority of Task 2B intact: code is still never evicted while any data line is present. The sensitivity study in Section 3.6 shows that RRPV 1 is the best of the four possible values, although RRPV 0 is very close.

Rule 3: prefetch hits are promoted like data hits. A prefetch request that hits in the L2 means SPP has just predicted a demand access to that line, so it is treated like a data hit: RRPV 0 in SRRIP mode and one step down in instruction mode. Leaving the RRPV unchanged on a prefetch hit (as some prefetch-aware RRIP proposals do) was measured to be slightly worse (Table 6).

The prefetch rule applies in the leader sets too. The leaders should measure the system as it actually runs, prefetches included. If they ignored prefetches (plain SRRIP insertion), the result was practically identical (1.0297 vs 1.0298 for the matching variants), so we keep one uniform rule.

Hardware cost. The type of the incoming request is already part of the request (MSHR entry), so the prefetch awareness needs no new storage. It needs only a comparator on the type field and a third insertion value. The total cost stays at the Task 2B figure of 50 bits on top of I-RRIP. The `l2_is_pf` bit array in the code feeds only the statistics in Table 1. It is not read by the policy and would not exist in hardware.

3.5 Results

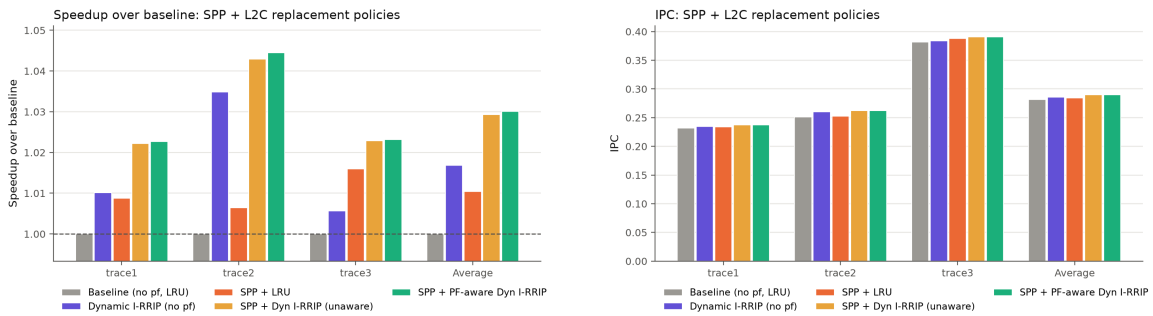
Table 3 and Table 4 report all required metrics, and Figures 1–3 plot them. The raw values are in `results/task3/task3_results.csv`, the tables in `results/task3/task3_tables.md`, and all logs in `results/task3/<configuration>/`.

Trace	Configuration	IPC	Speedup	Accuracy (%)	Coverage (%)	L1D load MPKI
Trace1	Baseline	0.2322	1.0000	–	–	10.679
	Dyn I-RRIP	0.2346	1.0101	–	–	10.680
	SPP + LRU	0.2343	1.0087	53.09	9.63	9.650
	SPP + Dyn (unaware)	0.2374	1.0222	52.93	9.53	9.661
	SPP + PF-aware Dyn	**0.2375**	**1.0226**	52.86	9.53	9.662
Trace2	Baseline	0.2513	1.0000	–	–	13.536
	Dyn I-RRIP	0.2601	1.0349	–	–	13.545
	SPP + LRU	0.2529	1.0065	44.98	2.67	13.175
	SPP + Dyn (unaware)	0.2621	1.0430	44.40	2.58	13.188
	SPP + PF-aware Dyn	**0.2625**	**1.0445**	44.50	2.57	13.189
Trace3	Baseline	0.3819	1.0000	–	–	3.403
	Dyn I-RRIP	0.3840	1.0057	–	–	3.404
	SPP + LRU	0.3880	1.0160	37.94	13.85	2.932
	SPP + Dyn (unaware)	0.3906	1.0229	37.85	13.76	2.935
	SPP + PF-aware Dyn	**0.3907**	**1.0232**	37.97	13.78	2.934
Average	Baseline	0.2814	1.0000	–	–	9.206
	Dyn I-RRIP	0.2861	1.0168	–	–	9.210
	SPP + LRU	0.2843	1.0104	45.34	8.72	8.586
	SPP + Dyn (unaware)	0.2897	1.0293	45.06	8.62	8.595
	SPP + PF-aware Dyn	**0.2899**	**1.0301**	45.11	8.63	8.595

Table 3: IPC, speedup over the baseline, and the L1D prefetch metrics. Accuracy and coverage exist only for the configurations with SPP.

Trace	Configuration	L2C load MPKI	L2 instr load MPKI	L2 data load MPKI	Avg. L2 miss latency (cyc.)	LLC load misses
Trace1	Baseline	17.703	10.368	7.334	140.5	307 044
	Dyn I-RRIP	19.042	9.072	9.970	107.1	292 496
	SPP + LRU	16.825	10.392	6.434	141.9	293 113
	SPP + Dyn (unaware)	17.873	9.043	8.830	108.2	277 907
	SPP + PF-aware Dyn	17.878	9.044	8.834	108.2	**277 422**
Trace2	Baseline	27.560	18.892	8.668	88.9	204 533
	Dyn I-RRIP	21.379	9.264	12.115	82.8	206 936
	SPP + LRU	27.486	19.132	8.354	89.4	196 869
	SPP + Dyn (unaware)	21.041	9.298	11.742	83.4	199 386
	SPP + PF-aware Dyn	**20.844**	**9.096**	11.748	83.7	199 287
Trace3	Baseline	3.960	1.124	2.837	156.9	78 933
	Dyn I-RRIP	3.994	0.727	3.267	109.3	74 944
	SPP + LRU	3.313	1.144	2.169	158.3	63 285
	SPP + Dyn (unaware)	3.308	0.725	2.583	113.1	58 942
	SPP + PF-aware Dyn	**3.307**	**0.723**	2.585	113.1	**58 886**
Average	Baseline	16.408	10.128	6.280	128.8	–
	Dyn I-RRIP	14.805	6.354	8.451	99.7	–
	SPP + LRU	15.875	10.222	5.652	129.8	–
	SPP + Dyn (unaware)	14.074	6.356	7.718	101.6	–
	SPP + PF-aware Dyn	**14.010**	**6.288**	7.722	101.7	–

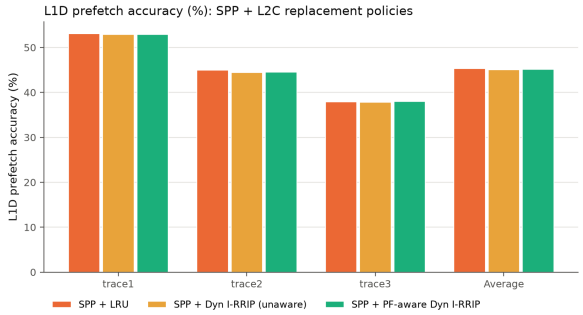
Table 4: L2 demand-load MPKI (total, instruction, data), with the average L2 miss latency and the LLC demand-load misses (loads that went to DRAM) as supporting metrics.



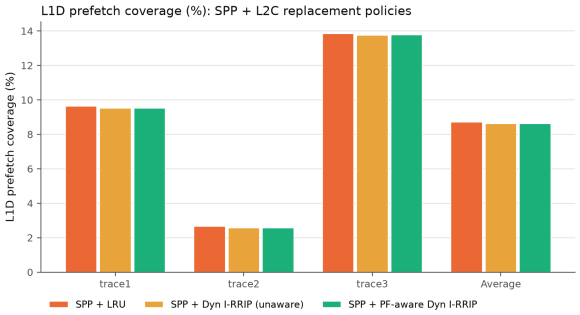
(a) Speedup over the baseline.

(b) IPC.

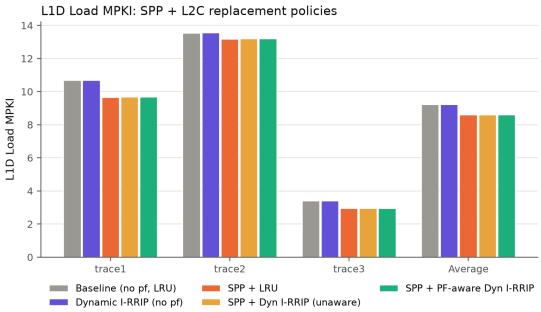
Figure 1: Performance of the five configurations.



(a) L1D prefetch accuracy.

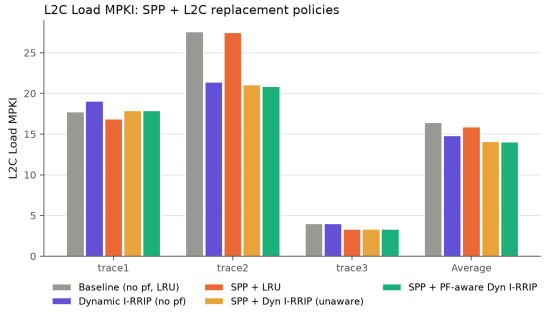


(b) L1D prefetch coverage.

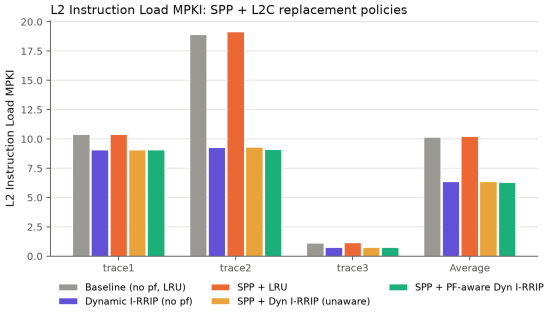


(c) L1D load MPKI.

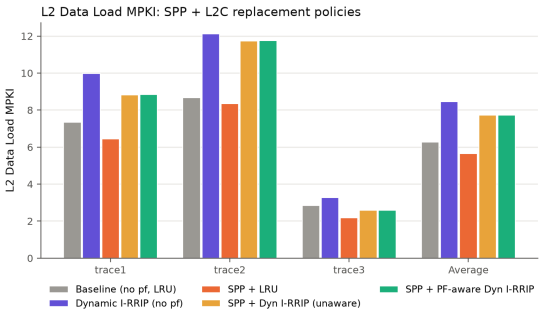
Figure 2: L1D prefetch metrics. The L2 policy hardly changes SPP's accuracy and coverage.



(a) L2C load MPKI.



(b) L2 instruction load MPKI.



(c) L2 data load MPKI.

Figure 3: L2 load MPKI, in total and split by line type.

3.6 Sensitivity study

Both parameters of the final design are read from environment variables, namely `DYN_IRRIP_PF_INS` and `DYN_IRRIP_THRESHOLD`, so one binary covers every point. Table 5 and Figure 4 vary one parameter at a time, keeping the other at its final value.

Value	Trace1	Trace2	Trace3	Geomean speedup	L2C load MPKI	L2 instr MPKI	L2 data MPKI
Prefetch fill insertion RRPV ($T = 0.05$)							
0	1.0226	1.0442	1.0231	1.0299	14.014	6.295	7.718
1 (final)	1.0226	1.0445	1.0232	**1.0301**	14.010	6.288	7.722
2 (= unaware)	1.0222	1.0430	1.0229	1.0293	14.074	6.356	7.718
3	1.0198	1.0430	1.0226	1.0284	14.078	6.361	7.717
Leader-set threshold T (prefetch insertion RRPV 1)							
0.05 (final)	1.0226	1.0445	1.0232	**1.0301**	14.010	6.288	7.722
0.10	1.0226	1.0441	1.0227	1.0297	14.012	6.291	7.721
0.20	1.0220	1.0395	1.0211	1.0275	14.032	6.418	7.613
0.35	1.0217	1.0383	1.0169	1.0256	14.132	6.558	7.574
0.50	1.0090	1.0366	1.0146	1.0200	13.843	7.124	6.719

Table 5: Sensitivity of SPP + prefetch-aware Dynamic I-RRIP. Speedup over the baseline; MPKI columns are means over the three traces. Insertion RRPV 2 with normal promotion is the unaware policy, so that row matches the unaware configuration exactly.

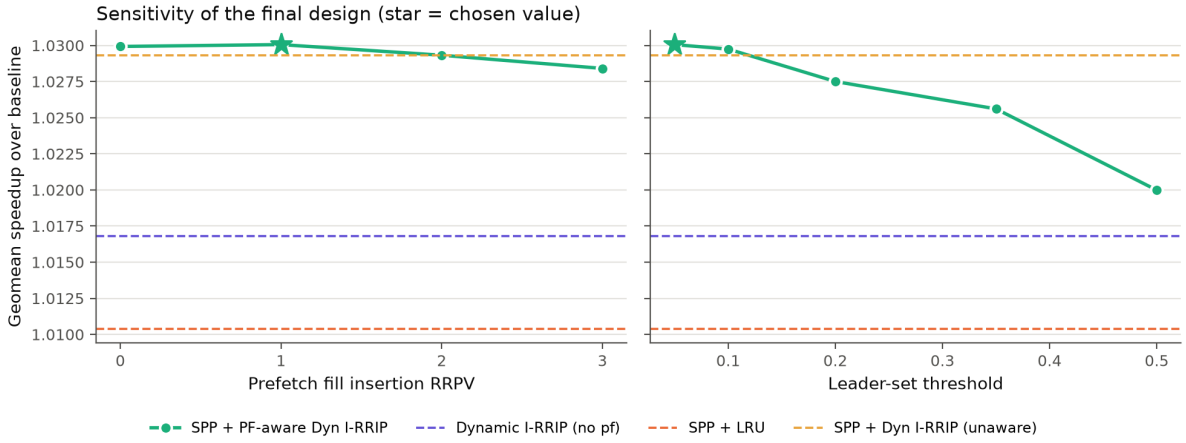


Figure 4: Geomean speedup of the final design against the prefetch insertion RRPV (left) and the leader-set threshold (right). The dashed lines are the other configurations.

Prefetch insertion. The speedup rises monotonically as prefetched lines are given more protection, from RRPV 3 to RRPV 1, and then falls slightly at RRPV 0. RRPV 0 ranks prefetched data equal to code in instruction mode, and the L2 instruction MPKI starts to rise again (6.288 $\rightarrow$ 6.295). The gain of RRPV 1 over the unaware policy (RRPV 2) shows mainly on Trace2: +0.14%, with a lower total L2 load MPKI (21.04 $\rightarrow$ 20.84). On Trace2 this comes from fewer instruction misses (9.30 $\rightarrow$ 9.10 MPKI); the data MPKI is unchanged. We did not isolate why a different treatment of data prefetches lowers instruction misses; presumably it is a second-order effect of the changed eviction order.

Threshold. The Task 2B conclusion still holds with SPP. The speedup falls monotonically as T grows, and $T = 0.05$ is the best value (or tied for best) on every trace. At $T = 0.50$, Trace1 ($r \approx 0.43$) spends 89% of its epochs in SRRIP mode and loses most of its gain (1.0226 $\rightarrow$ 1.0090). Trace3 ($r \approx 0.17$) spends 33–67% of its epochs in SRRIP from $T = 0.20$ upwards. At the chosen $T = 0.05$, all three traces stay in instruction mode for the whole measured region.

3.7 Analysis

The goal is met on every trace. SPP + prefetch-aware Dynamic I-RRIP has the highest IPC on all three traces. The geometric-mean speedup is 1.0301: +1.31% over Dynamic I-RRIP alone (1.0168) and +1.95% over standalone SPP (1.0104). Per trace, it is +1.2%, +0.9% and +1.7% over Dynamic I-RRIP, and +1.4%, +3.8% and +0.7% over SPP.

The two mechanisms are complementary. The product of the two individual speedups is $1.0168 \times 1.0104 = 1.0274$, and the combination reaches 1.0293 even without awareness, so the benefits add up rather than overlap. They remove different misses:

- SPP removes L1D data misses (coverage 2.6–13.8%) and does not change the L2 instruction MPKI.
- Dynamic I-RRIP cuts the L2 instruction MPKI by 38% on average (10.22 $\rightarrow$ 6.29 with SPP). It does this at the cost of L2 data misses (5.65 $\rightarrow$ 7.72 MPKI with SPP).
- SPP softens that cost. With SPP, the L2 data MPKI under Dynamic I-RRIP is 7.72, compared with 8.45 without a prefetcher. That reduction (0.73) is slightly larger than what SPP achieves under LRU (6.28 $\rightarrow$ 5.65).

The L2 miss latency stays at Dynamic I-RRIP's low level, 101.7 cycles against 129.8 for SPP + LRU. LLC load misses (DRAM trips) are the lowest of all configurations on Trace1 and Trace3.

The prefetch awareness adds a small but consistent gain. Compared with SPP plus the unaware Task 2B policy, the prefetch-aware policy is faster on every trace: Trace1 +0.04%, Trace2 +0.14%, Trace3 +0.03%, geomean 1.0293 $\rightarrow$ 1.0301. It also has fewer LLC load misses on every trace, and lower total L2C and instruction MPKI on average. The gain is small for the reason found in Section 3.3: SPP sends only 1.1–2.6 prefetches per kilo-instruction to the L2, and 98% of their L2 copies are never demanded there. No rule for those lines can change much. The sensitivity and exploration results show that the *direction* of the rule matters more than its size. A policy that guesses wrongly in either direction is clearly worse than the unaware one: dead-on-arrival insertion loses 0.09%, and pinning prefetches loses 0.56%.

SPP's accuracy and coverage are almost independent of the L2 policy. Accuracy and coverage change by at most 0.6 and 0.1 percentage points between the three SPP configurations (Table 3). SPP trains on L1D accesses and throttles on its own L1D accuracy, so the L2 policy changes when a prefetch completes (L2 hit or LLC access), not which lines are prefetched. The speedup of the combined design therefore comes from the L2 side, as intended.

3.8 Design exploration

Before fixing the design, we evaluated a range of prefetch-handling options on all three traces with an exploration build of the policy, in which every option sits behind an environment variable

(Table 6).

Prefetch handling (SPP + Dyn I-RRIP, $T = 0.05$)	Trace1	Trace2	Trace3	Geomean
Unaware (Task 2B policy)	1.0222	1.0430	1.0229	1.0293
Fill RRPV 3, prefetch hit not promoted	1.0196	1.0429	1.0225	1.0283
Fill RRPV 3, prefetch hit promoted	1.0198	1.0430	1.0226	1.0284
Fill RRPV 2, prefetch hit not promoted	1.0219	1.0430	1.0229	1.0292
Fill RRPV 2, unused prefetches at RRPV 3 evicted first	1.0218	1.0428	1.0228	1.0291
Adaptive: RRPV 2/3 by L2 usefulness of recent prefetch fills	1.0199	1.0429	1.0226	1.0284
Fill RRPV 1, prefetch hit not promoted	1.0220	1.0443	1.0232	1.0298
with leader sets ignoring prefetches	1.0218	1.0444	1.0232	1.0297
Fill RRPV 1, prefetch hit promoted (final)	1.0226	1.0445	1.0232	**1.0301**
adaptive: RRPV 3 when recent L1D accuracy $< 40\%$	1.0225	1.0443	1.0228	1.0298
unused prefetches never evicted while other data exists	1.0132	1.0402	1.0173	1.0235
Fill RRPV 0, prefetch hit not promoted	1.0227	1.0444	1.0229	1.0299
Fill RRPV 0, prefetch hit promoted	1.0226	1.0442	1.0231	1.0299

Table 6: Prefetch-handling variants, as speedup over the baseline. Raw numbers, the legend and all logs are in `results/task3/design_exploration/`. The final design was also re-run with a rebuilt binary, and the results were identical.

Adaptive insertion did not pay off. A natural idea is to set the insertion depth from the measured usefulness of prefetches. Measured at the L2 (prefetched lines that get a demand hit in the L2), the usefulness is only 1–2% (Table 1) and never reached the 25% threshold in any epoch, so the adaptive policy always picks RRPV 3 and behaves like dead-on-arrival insertion. Measured at the L1D (SPP's own accuracy, 38–53% on average), the per-epoch accuracy hovers around a 40% threshold: it is above it in 86%, 42% and 39% of the epochs on Trace1–3. The policy then alternates between RRPV 1 and RRPV 3 and lands between the two fixed policies (1.0298). On these traces neither signal separates good phases from bad ones, so a fixed insertion depth is enough. Adaptive insertion would matter for a prefetcher whose accuracy swings between phases. SPP already throttles its own degree by accuracy (Task 1B), which reduces the need for it.

Writebacks. For completeness we also tried inserting writeback fills at RRPV 3. Dynamic I-RRIP turns many L1D writebacks into L2 misses (68.9% writeback miss rate on Trace2, against 3.0% for SPP + LRU), most likely because data lines are evicted first in instruction mode. This lowered the speedup to 1.0286, so writebacks keep the Task 2B treatment. It is not part of the prefetch handling.

3.9 Limitations

The prefetch-aware rules add only +0.08% geomean on top of the plain combination, because little prefetch traffic reaches the L2 with this L1D prefetcher. The rules would matter more for an L2 prefetcher or a more aggressive L1D prefetcher, but we could not evaluate those within the assignment's configuration. The insertion RRPV and the threshold were chosen on the same three traces they are evaluated on, so the measured gains may be optimistic for unseen workloads. The small instruction-MPKI improvement on Trace2 from prefetch-aware insertion was observed but not explained mechanistically.

3.10 Reproducing the results

```
./build_champsim.sh no  lru       baseline
./build_champsim.sh no  dyn_irrip dyn_irrip
./build_champsim.sh spp lru       spp
./build_champsim.sh spp dyn_irrip spp_dyn_irrip
# the provided traces are gzip files without an extension
mkdir -p traces
for i in 1 2 3; do ln -sf ../Trace$i-00$i traces/trace$i.champsimtrace.gz; done
task3/run_task3.sh traces 12     # 3 traces x (5 configs + 7 sensitivity)
python3 task3/parse_results.py  # task3_tables.md, task3_results.csv
python3 task3/plot_results.py   # results/task3/plots/
# single run; DYN_IRRIP_PF_AWARE=0 gives the unaware
# Task 2B behaviour, DYN_IRRIP_PF_INS sets the RRPV
DYN_IRRIP_PF_INS=1 bin/<spp_dyn_irrip binary> \
    -warmup_instructions 25000000 \
    -simulation_instructions 25000000 \
    -traces traces/trace1.champsimtrace.gz
```